

BP_GameManager

1. BP_GameManager Overview

Purpose

BP_GameManager is the core manager Blueprint of the VR game. It receives the rule configuration, controls the game flow (start, playing, end), manages the score, spawns and destroys weapons, handles the countdown and attempt counting, and communicates with the TriggerArea, Weapon, and Target Blueprints through the BPI_GameManager interface.

Design

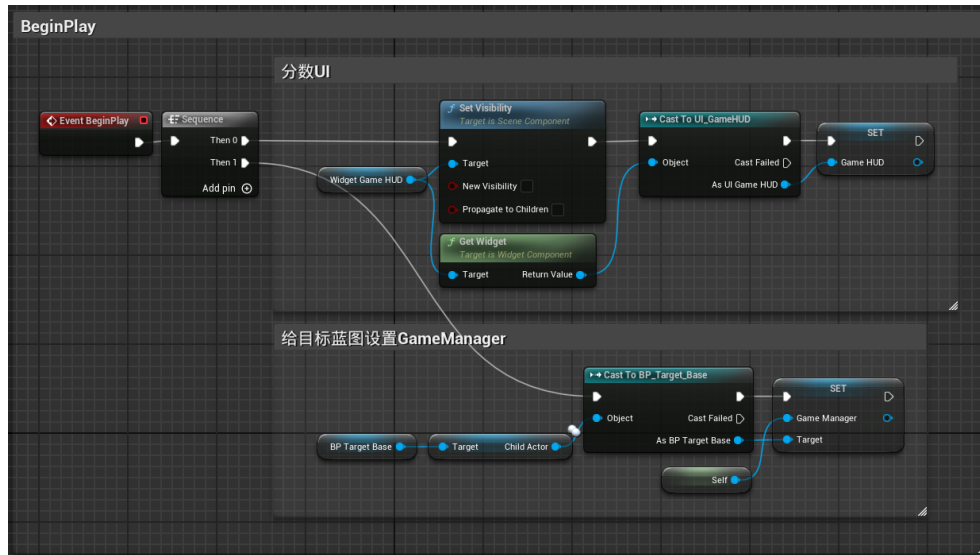
- Dual scoring modes: supports ByCount (attempt-based) and ByTime (countdown-based), branching through a Switch
- Delayed ending: whether the game ends is only checked when a weapon is released (not immediately when the count hits zero), giving the player buffer time
- Interface driven: exposes StartGame, EndGame, AddScore, ConsumeAttempt and other methods through the BPI_GameManager interface for external calls

2. BeginPlay Initialization

Description

After the level loads, initializes the score UI and target references, establishing two-way references between the GameManager and the Targets.

Blueprint screenshots



Key points

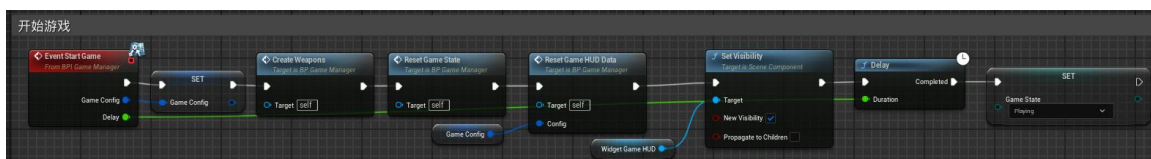
- Target instances are obtained through Child Actor Components; after the Cast, the GameManager reference is set, establishing two-way communication
- The score UI is initialized and its reference cached in BeginPlay, avoiding repeated Get Widget calls at runtime

3. StartGame

Description

Receives the game configuration from the TriggerArea, initializes the weapons, state, and HUD in order, and finally sets the game state to Playing.

Blueprint screenshots



Key points

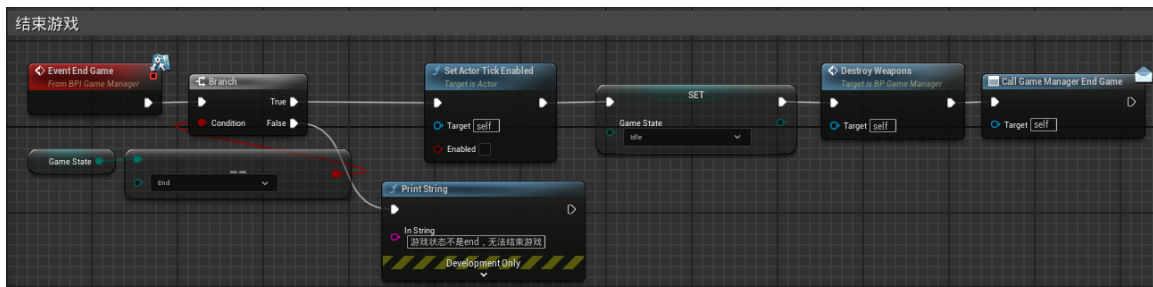
- The Delay comes before setting Playing, ensuring weapons and HUD are ready

4. EndGame

Description

After validating the game state, runs the cleanup flow: disable Tick, reset the state, destroy weapons after a delay, and broadcast the end event.

Blueprint screenshots



Key points

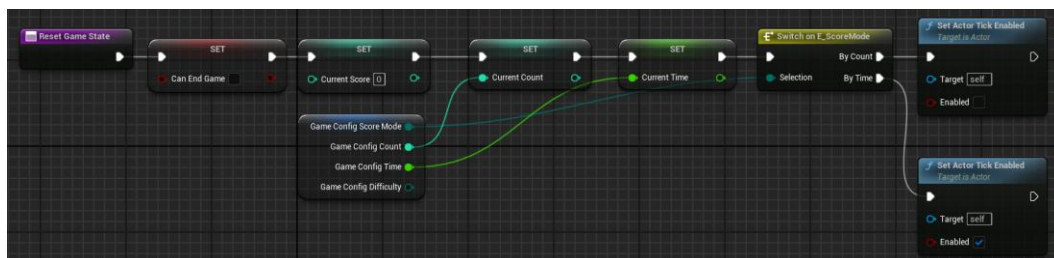
- The condition requires Game State == End, preventing the game from ending in an invalid state
- Before calling EndGame, the TriggerArea must first SET Game State = End, otherwise the guard fails
- The 2-second delay gives the player buffer time and waits for in-flight weapons to finish
- Call Game Manager End Game broadcasts an event; the TriggerArea's SetEndGameEvent responds by restoring UI interaction and the Start button

5. ResetGameState

Description

Resets the runtime game variables and toggles Tick based on the scoring mode for performance.

Blueprint screenshots



•

Key points

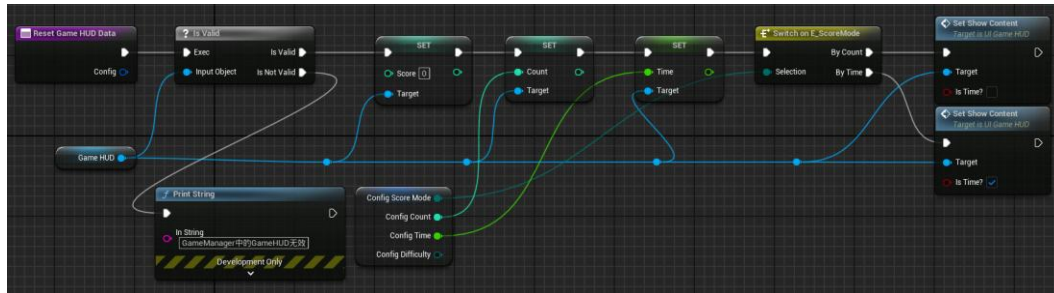
- ByCount mode does not need Tick; disabling it reduces per-frame cost
- ByTime mode needs Tick to drive the countdown; once enabled, Current Time decreases every frame

6. ResetGameHUDData

Description

Resets the HUD's score, count, and time displays to their initial values and switches the displayed content based on the scoring mode.

Blueprint screenshots



Key points

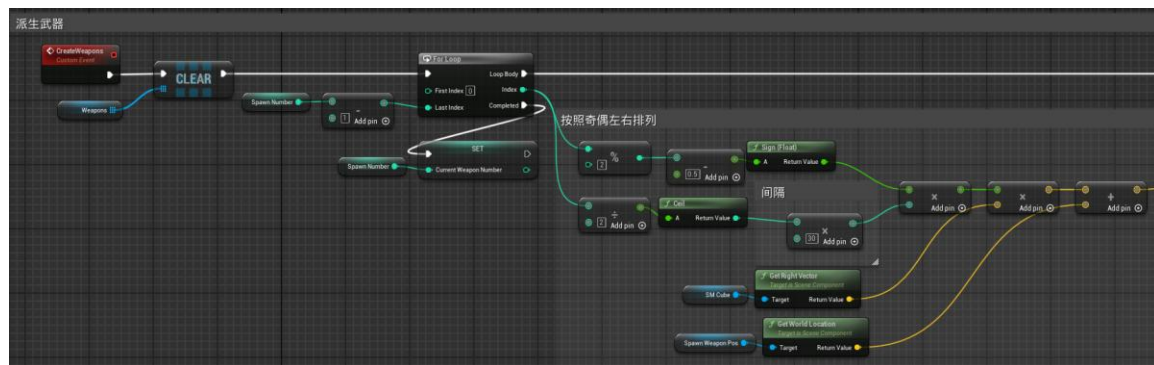
- Is Valid prevents crashes when the HUD is not initialized
- Set Show Content switches the HUD display by scoring mode (ByCount shows remaining attempts, ByTime shows the countdown)

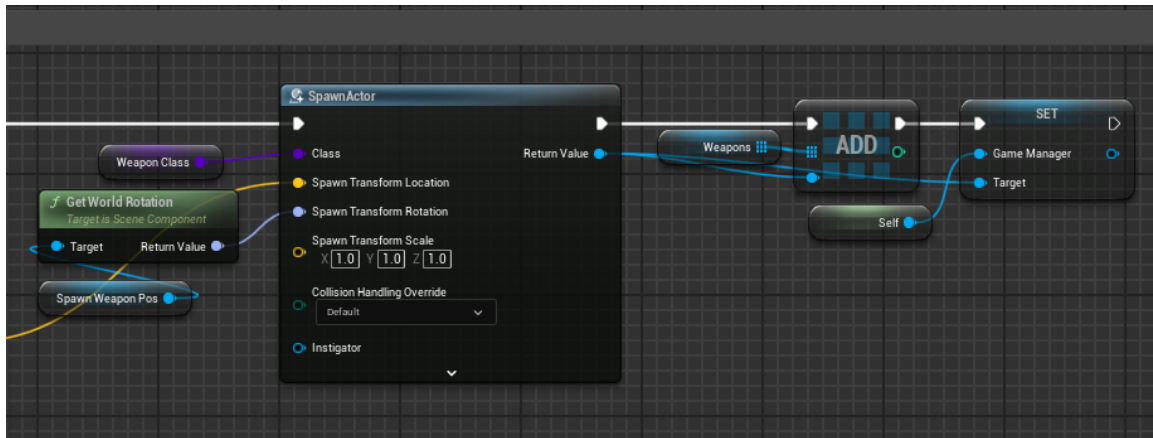
7. CreateWeapons

Description

Spawns the requested number of weapon instances, arranging them left and right of the spawn point with an odd/even alternating algorithm, and tracks all weapons in an array.

Blueprint screenshots





Arrangement algorithm

Index	Parity	Offset Direction	Position
0	Even	Left	Base + 0 × spacing × (-1)
1	Odd	Right	Base + 1 × spacing × (+1)
2	Even	Left	Base + 2 × spacing × (-1)
3	Odd	Right	Base + 3 × spacing × (+1)

Key points

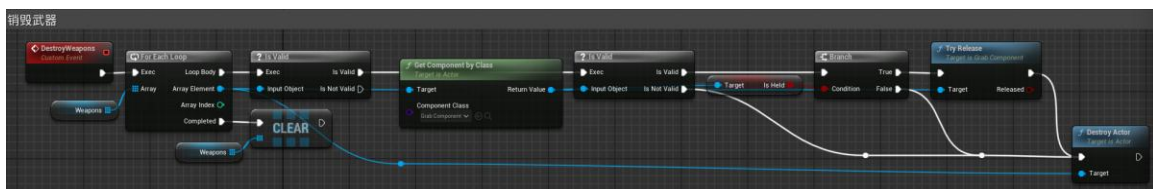
- WeaponClass is the weapon parent class; each game can derive subclasses and pick the weapon subclass in the editor
- The odd/even alternating arrangement distributes weapons symmetrically on both sides of the spawn point

8. DestroyWeapons

Description

Iterates the weapon array, safely destroys all weapon instances, and clears the array; if a weapon is currently held by the player, it is released first before being destroyed.

Blueprint screenshots



Key points

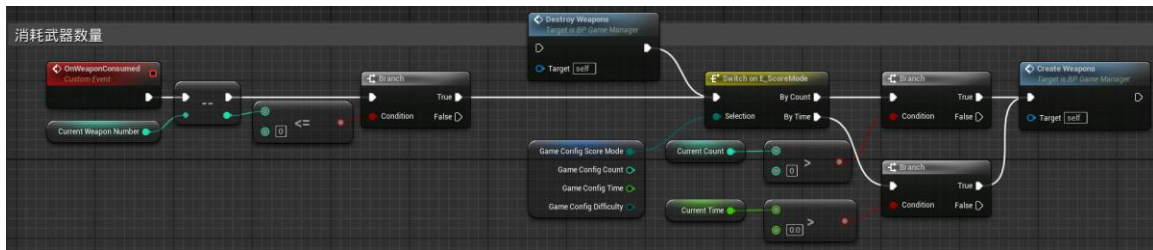
- The Is Valid check prevents destroying Actors that no longer exist (e.g. weapons that already destroyed themselves after hitting a target)
- CLEAR ensures the array keeps no dangling references, preparing a clean state for the next CreateWeapons

9. OnWeaponConsumed

Description

Triggered when a weapon is released; decrements the in-scene weapon count, and when it reaches zero destroys leftover weapons and decides by the rules whether to restock new weapons.

Blueprint screenshots



Key points

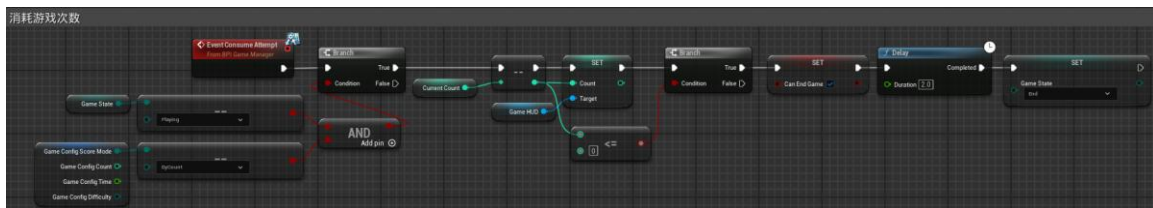
- CurrentWeaponNumber tracks the number of weapons in the scene; it is a different counter from Current Count (total game attempts)
- When it reaches zero, DestroyWeapons clears the field first and then CreateWeapons restocks, avoiding old and new weapons overlapping
- The Switch branch ensures that when the game is about to end (Count/Time at zero) no more weapons are restocked

10. ConsumeAttempt

Description

Consumes one game attempt when a weapon is released, updates the HUD, and sets the end flag and state when the count reaches zero.

Blueprint screenshots



Key points

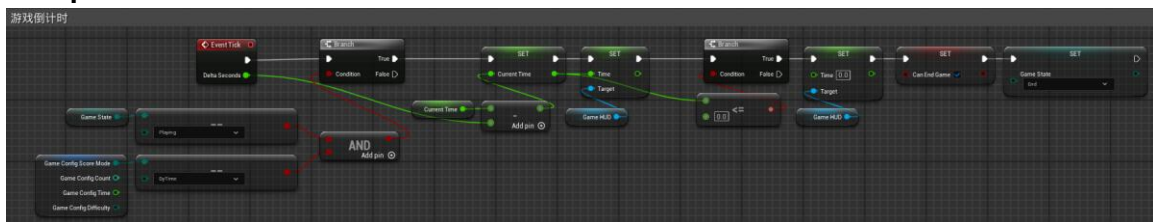
- The condition ensures attempts are only consumed while Playing in ByCount mode
- A 2-second delay lets the last throw wait to see whether it hits a target before the game ends

11. Tick Countdown

Description

In ByTime mode, the remaining time decreases every frame; when it reaches zero the end flag and state are set. Mutually exclusive with ConsumeAttempt.

Blueprint screenshots



Key points

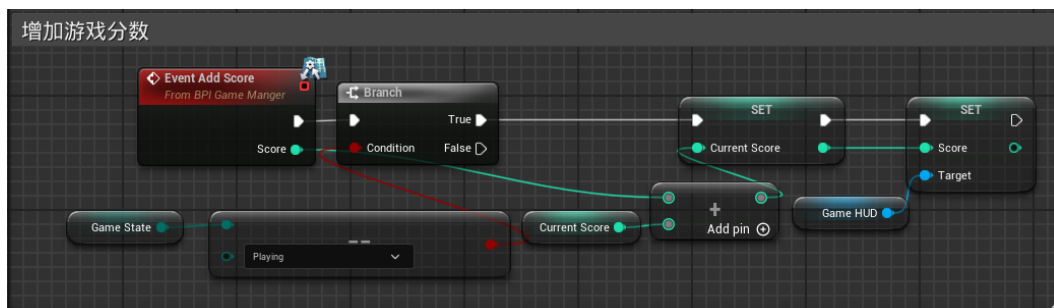
- The countdown only runs in ByTime mode while Playing; Tick was already enabled in ResetGameState
- In ByCount mode Tick is disabled and the countdown logic never runs
- Both modes trigger the same ending flow through Game State = End

12. AddScore

Description

Reports the score when a target is hit, adds it to the total, and updates the HUD. Only responds while Playing.

Blueprint screenshots



Key points

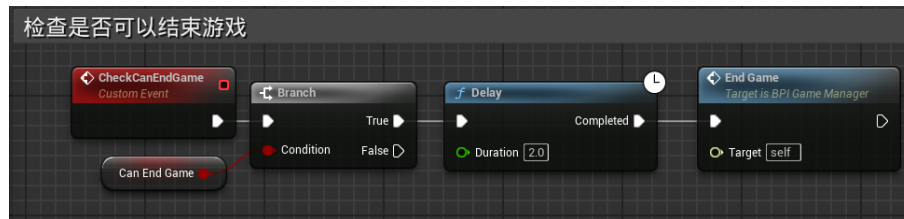
- Checking the game state prevents illegal scoring outside of gameplay
- The Score parameter is passed by the Target; different targets can pass different values (implemented per game)
- After scoring, the HUD is updated in sync so the data and display stay consistent

13. CheckCanEndGame

Description

After a weapon is released, checks whether the end condition is met and calls EndGame if so. Implements the delayed-ending mechanism of "only deciding the end when the weapon is released".

Blueprint screenshots



Key points

- Can End Game is set to true by ConsumeAttempt (attempts at zero) or Tick (time exhausted)
- It is only called when the weapon is released (inside Shoot), not immediately when count/time hits zero, giving the player buffer

14. BP_Weapon_Base Overview

Purpose

BP_Weapon_Base is the VR weapon base class Blueprint. It uses a GrabComponent for grabbing/releasing, triggers the attack logic (Shoot) on release, and destroys itself after hitting a target. Subclasses override Shoot to implement different attack styles (projectile flight, force-driven rolling, melee collision, launched projectiles, etc.).

Design

- Event driven: OnDropped (release) → Shoot (attack) → ConsumeAttempt + CheckCanEndGame (game management) → OnWeaponConsumed (weapon management)
- Shoot is a virtual function: the base class provides the call framework, subclasses override it with their attack logic
- Self-destruct on hit: HitTarget triggers DestroyActor, suited to one-shot projectile weapons

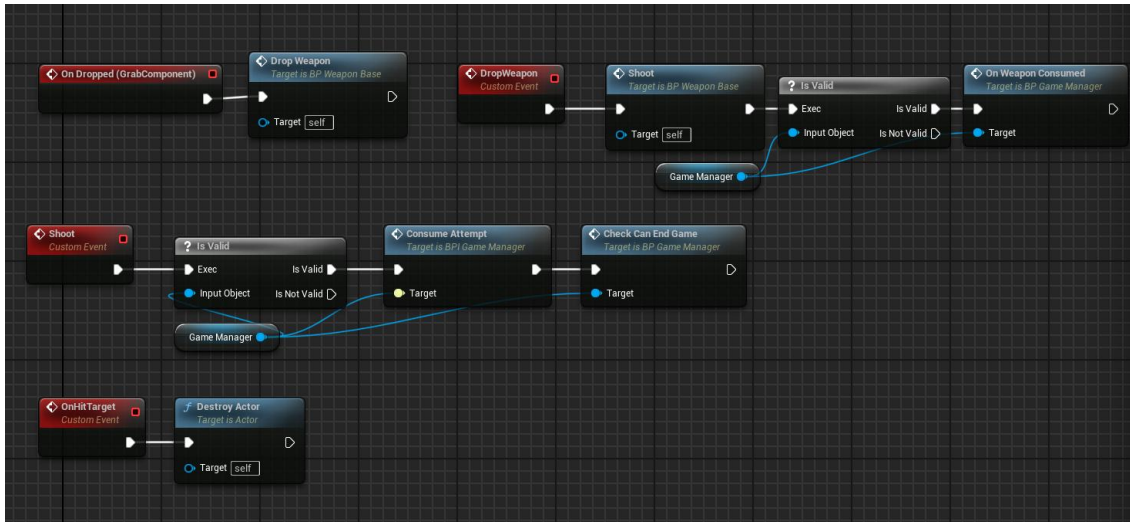
- GameManager reference: interface methods are called directly through the GameManager variable, no event dispatcher needed

15. BP_Weapon_Base Logic

Description

Three independent event chains: OnDropped notifies the GameManager after release, Shoot performs the attack and game-management calls, and HitTarget handles self-destruction after a hit.

Blueprint screenshots



- OnDropped (weapon release event)
- Shoot (attack custom event)
- HitTarget (target hit event)

Execution order

OnDropped → Shoot (executes ConsumeAttempt + CheckCanEndGame) → after returning → OnWeaponConsumed. Shoot is a synchronous call; once it finishes, the rest of OnDropped's nodes continue.

Key points

- In the base class, Shoot only makes the framework call; subclasses overriding it call the Parent Node first (keeping the base logic) before adding their attack
- The DestroyActor in HitTarget suits one-shot weapons (darts, arrows); melee weapon subclasses can override it to not destroy

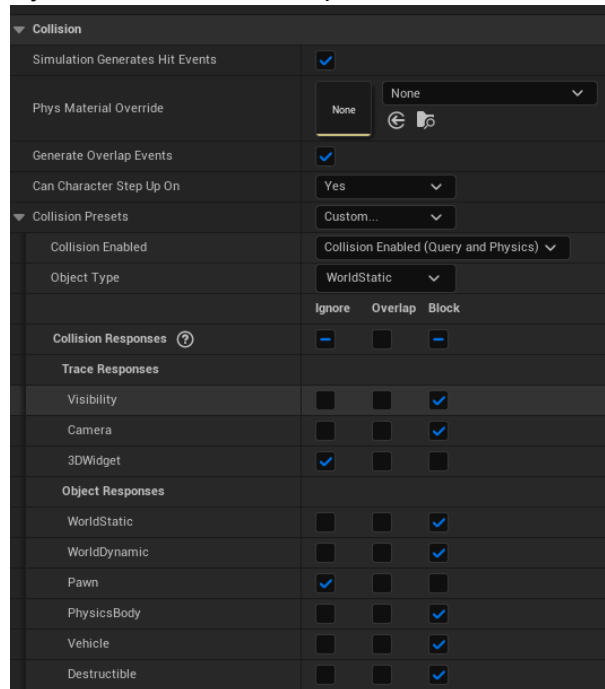
16. BP_Target_Base Overview

Purpose

BP_Target_Base is the VR target base class Blueprint. It handles what happens after being hit by a weapon: validates the weapon type, notifies the weapon of the hit, plays a sound, and reports the score to the GameManager. Subclasses can override it for different target behaviors (different values, different feedback effects, etc.).

Design

- Type safety: Cast To BP_Weapon_Base verifies the hitter is a weapon, preventing non-weapon objects from falsely triggering scoring
- Event delegation: OnHit notifies the weapon's HitTarget event, letting the weapon decide whether to destroy itself
- Collision isolation: Pawn = Ignore keeps VR character collisions from disturbing the target, PhysicsBody = Block ensures weapons can hit it

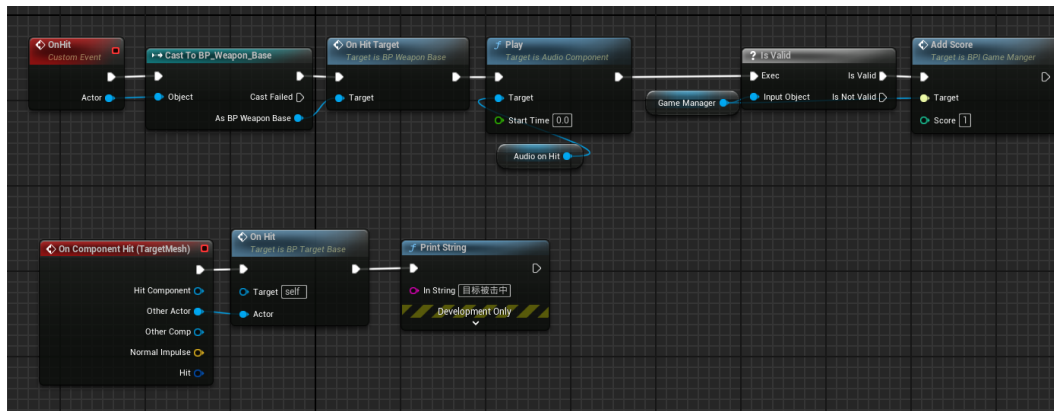


17. BP_Target_Base Logic

Description

OnHit runs the full post-hit flow (validate → notify → sound → score); On Component Hit is the physics collision entry point and contains debug output.

Blueprint screenshots



Key points

- OnHit and On Component Hit are separate: On Component Hit is the physics event entry, OnHit is the logic; subclasses overriding OnHit need not care about physics event binding
- When the Cast fails nothing is processed, avoiding environment objects falsely triggering scoring